

AIMS AI Agent — Enterprise Architecture

Alpha Integrated Management System AI

Version: 2.0 (Architecture Draft)
Date: July 2026
Status: Pre-Implementation Design

Table of Contents

- 1. Executive Summary
- 2. System Overview
- 3. Design Principles
- 4. Architecture Layers
- 5. RAG Pipeline
- 6. Knowledge Ingestion
- 7. Agent Orchestration
- 8. ERPNext Integration
- 9. Database Schema
- 10. API Contracts
- 11. Security Model
- 12. Multi-Language Design
- 13. Frontend Architecture
- 14. Infrastructure
- 15. Migration Path

1. Executive Summary

AIMS AI Agent is a production-grade, enterprise AI assistant for companies operating in Tanzania. It integrates with ERPNext/Frappe Framework and serves as a professional consultant capable of handling accounting, finance, taxation, auditing, payroll, ERP, and compliance questions.

The system uses a Retrieval-Augmented Generation (RAG) architecture where every answer is grounded in trusted knowledge sources. It never hallucinates laws, sections, or accounting standards.

Current State (Alpha)

- Next.js 15 App Router with TypeScript
- Static knowledge base: 9 entries (TRA/PDPC sources)
- Keyword-based retrieval with Swahili synonym expansion
- Deterministic calculators: PAYE, VAT, SDL, WHT
- PDF upload and text extraction

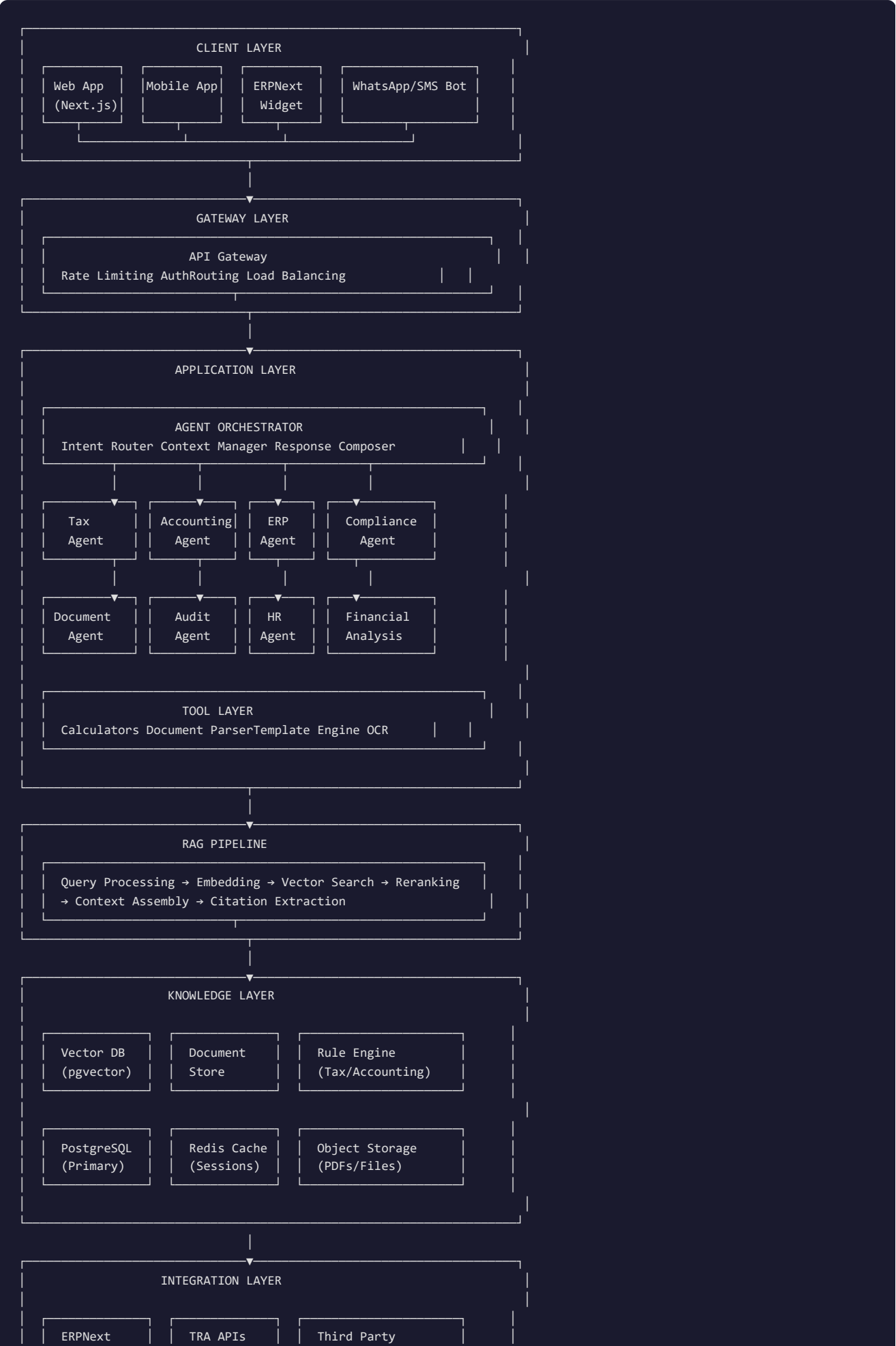
- Swahili language detection
- No LLM integration
- No database (JSON file-based)
- No authentication
- No ERPNext integration

Target State (Production)

- Multi-agent AI system with centralized knowledge
 - Vector embeddings with semantic search
 - LLM-powered responses with source grounding
 - PostgreSQL + pgvector for persistence
 - ERPNext/Frappe bidirectional integration
 - Role-based access control
 - Multi-tenant support
 - Audit trail and compliance logging
 - Document understanding pipeline
 - Financial analysis engine
-

2. System Overview

2.1 High-Level Architecture





2.2 Technology Stack

LayerTechnologyPurpose

FrontendNext.js 15, TypeScript, Tailwind CSSWeb application
APINext.js API Routes + tRPCType-safe API layer
OrchestrationCustom agent frameworkMulti-agent coordination
LLMOpenAI GPT-4 / Claude / Local modelsText generation
EmbeddingsOpenAI text-embedding-3-smallVector embeddings
Vector DBpgvector (PostgreSQL extension)Semantic search
Primary DBPostgreSQL 16Persistent storage
CacheRedis 7Session management, caching
Object StorageMinIO / S3Document storage
ERPNextFrappe FrameworkERP integration
Message QueueBullMQ (Redis-backed)Background jobs
MonitoringOpenTelemetry + GrafanaObservability

3. Design Principles

3.1 Core Principles

1. **Source Grounding:** Every tax/legal/compliance answer must cite an official source. No hallucination.
2. **Deterministic Calculations:** All tax calculations use coded functions, not LLM arithmetic. The LLM explains results, never computes them.
3. **Conservative by Default:** When uncertain, the system asks clarifying questions or recommends human review rather than guessing.
4. **Audit Trail:** Every interaction, calculation, document access, and recommendation is logged and traceable.
5. **Multi-Tenant Isolation:** Company data is strictly isolated. No cross-tenant data leakage.
6. **Role-Based Access:** Users see only what their role permits. Sensitive data (TIN, salaries, bank details) is protected.
7. **Modular Agents:** Each agent is independently deployable but shares the centralized knowledge architecture.
8. **Bilingual Native:** English and Swahili are first-class citizens, not afterthoughts.
9. **ERPNext-First:** When ERPNext can handle a task, direct users there. AIMS supplements, not replaces.
10. **Progressive Disclosure:** Simple answers for simple questions. Deep analysis when requested.

3.2 Anti-Patterns to Avoid

- Never fabricate legal references, section numbers, or act names
- Never perform arithmetic in the LLM response (use calculators)
- Never expose internal prompts or system instructions

- Never store sensitive data (TIN, NIDA, salaries) in chat logs without encryption
 - Never provide final tax opinions without human review for high-risk matters
 - Never bypass the knowledge base to answer compliance questions
-

4. Architecture Layers

4.1 Client Layer

The client layer handles all user-facing interfaces.

Web Application (Next.js)

- Server-side rendered for performance
- Responsive design (desktop, tablet, mobile)
- Dark mode support
- Real-time streaming for LLM responses

ERPNext Widget

- Embedded as a Frappe custom page or dialog
- Communicates via Frappe whitelisted APIs
- Inherits ERPNext session and permissions
- Context-aware (knows which DocType is open)

WhatsApp/SMS Bot (Future)

- Simplified interface for field workers
- Template-based responses for common queries
- Compliance deadline reminders

4.2 Gateway Layer

API Gateway Responsibilities

- Request routing and load balancing
- Rate limiting (per user, per tenant)
- Authentication token validation

- Request/response logging
- CORS management
- WebSocket connection management for streaming

4.3 Application Layer

The application layer contains the agent orchestration system. See Section 7 for detailed agent design.

4.4 RAG Pipeline

See Section 5 for detailed RAG architecture.

4.5 Knowledge Layer

The knowledge layer stores and manages all data.

PostgreSQL (Primary)

- User accounts and roles
- Company/tenant data
- Conversation history
- Audit logs
- Tax rules and versions
- Document metadata

Compliance calendar

pgvector (Vector Store)

- Document chunk embeddings
- Knowledge base embeddings
- Conversation context embeddings

Redis (Cache)

- Session management
- Active conversation context
- Rate limiting counters
- Frequently accessed tax rules

Object Storage (MinIO/S3)

- Uploaded PDFs and documents
- Generated reports
- Export files

4.6 Integration Layer

ERPNext/Frappe Integration

- Bidirectional data sync via Frappe API
- Custom Frappe app for AIMS hooks
- DocType for AIMS conversations and audit logs
- Webhook listeners for ERPNext events

External APIs (Future)

- TRA e-filing integration
- Bank statement imports
- Government gazette feeds

5. RAG Pipeline

5.1 Pipeline Architecture



5.2 Query Processing Details

Language Detection

```

type Language = "en" "sw" "mixed";

function detectLanguage(query: string): Language {
  // Swahili keyword detection
  // Mixed language detection
  // Default to English
}

```

Intent Classification

The system classifies queries into intents to route to appropriate agents and retrieval strategies:

IntentAgentRetrieval Priority

```

tax_question Tax AgentTax Acts, Finance Acts, TRA Rulings
tax_calculation Calculator ToolRule engine only
accounting_question Accounting AgentIFRS/IAS, Accounting standards
erpnext_help ERP AgentERPNext docs, Frappe docs
compliance_check Compliance AgentCompliance calendar, Regulations
document_draft Document AgentTemplates, Previous drafts
audit_question Audit AgentISA, Audit standards
payroll_question HR AgentLabour laws, NSSF, WCF
financial_analysis Analysis AgentFinancial reporting standards
general Tax Agent (fallback)All sources

```

Entity Extraction

Extract key entities from queries:

- Tax types (VAT, PAYE, SDL, WHT, CIT)
- Amounts and dates
- Company names and TINs
- Section references
- Document types

5.3 Retrieval Strategy

Hybrid Search with Reciprocal Rank Fusion

```

type RetrievalResult = {
  chunk: KnowledgeChunk;
  vectorScore: number;
  bm25Score: number;
  fusedScore: number;
  source: string;
};

function hybridRetrieve(
  query: string,
  options: RetrievalOptions
): RetrievalResult[] {
  const vectorResults = vectorSearch(queryEmbedding, options.k * 2);
  const bm25Results = bm25Search(queryTokens, options.k * 2);

  // Reciprocal Rank Fusion
  const fused = reciprocalRankFusion(
    vectorResults,
    bm25Results,
    options.k
  );

  return fused;
}

function reciprocalRankFusion(
  resultSets: RetrievalResult[][],
  k: number
): RetrievalResult[] {
  const scores = new Map();
  const K = 60; // RRF constant

  for (const results of resultSets) {
    results.forEach((result, rank) => {
      const id = result.chunk.id;
      const current = scores.get(id) ?? 0;
      scores.set(id, current + 1 / (K + rank + 1));
    });
  }

  return Array.from(scores.entries())
    .sort((a, b) => b[1] - a[1])
    .slice(0, k);
}

```

5.4 Embedding Strategy

Chunk Configuration

ParameterValueRationale

Chunk size512 tokensBalance between context and precision

Chunk overlap64 tokensMaintain context across boundaries

Embedding modeltext-embedding-3-smallCost-effective, good performance

Embedding dimensions1536Standard for OpenAI embeddings

Embedding Generation Pipeline

1. Document ingestion → Text extraction → Cleaning
2. Smart chunking (respect section boundaries)
3. Metadata attachment (source, date, category, jurisdiction)
4. Batch embedding generation
5. Vector storage with metadata
6. Index creation for efficient search

5.5 Citation System

Every response must include verifiable citations.

Citation Format

```
type Citation = {  
  id: string;  
  title: string;  
  sourceOrg: "TRA" "Ministry of Finance" "IFRS Foundation" ...;  
  sourceUrl: string;  
  section?: string;           // e.g., "Section 12(1)(a)"  
  effectiveDate?: string;  
  lastReviewed: string;  
  confidence: "high" "medium" "low";  
  chunkIds: string[];         // Which chunks were used  
};
```

Citation Rules

1. Minimum 1 citation for any tax/legal claim
2. No citation = no answer (state "source not available")
3. Conflicting sources = flag for human review
4. Outdated sources = clearly mark the date

6. Knowledge Ingestion

6.1 Ingestion Pipeline



6.2 Document Categories

Category	Examples	Priority
tax_act	Income Tax Act, VAT Act, Tax Admin Act	Critical
finance_act	Finance Acts (2020-2026)	Critical
tra_guidance	Practice Notes, Public Rulings	High

accounting_standard	IFRS, IAS, IPSAS	High
labour_law	ELRA, NSSF, WCF regulations	High
company_law	Companies Act, BRELA regulations	Medium
erpnext_doc	ERPNext user guides, API docs	Medium
company_policy	Internal SOPs, policies	Medium
audit_standard	ISA standards	Medium
uploaded_document	User-uploaded PDFs	Varies

6.3 Quality Control

Automated Checks

- Text extraction completeness (>90% of pages)
- Chunk size within bounds
- Metadata completeness
- Embedding generation success
- Duplicate detection

Human Review

- Admin approval required before publication
- Quality scoring (1-5 scale)
- Source authority verification
- Effective date verification

7. Agent Orchestration

7.1 Orchestrator Design

The Agent Orchestrator is the central coordination point.



7.2 Agent Definitions

Tax Advisory Agent

- Primary agent for all tax-related queries
- Access to: Tax acts, Finance acts, TRA guidance
- Tools: PAYE calculator, VAT calculator, SDL calculator, WHT calculator, CIT calculator
- Confidence thresholds: High (>0.85), Medium (>0.6), Low (<0.6)

Accounting Agent

- Handles bookkeeping, financial reporting, IFRS/IAS questions
- Access to: IFRS standards, IAS standards, IPSAS standards
- Tools: Financial ratio calculator, Depreciation calculator

ERPNext Agent

- Guides users through ERPNext functionality

- Access to: ERPNext docs, Frappe framework docs
- Tools: ERPNext API reference, Workflow designer

Compliance Agent

- Tracks deadlines, filing obligations, regulatory requirements
- Access to: Compliance calendar, Regulatory deadlines
- Tools: Compliance calendar generator, Reminder scheduler

Document Agent

- Drafts letters, memos, checklists, objection drafts
- Access to: Document templates, Previous drafts
- Tools: Template engine, PDF/DOCX generator

Audit Agent

- Internal audit support, fraud detection, control assessment
- Access to: ISA standards, Audit checklists
- Tools: Anomaly detector, Control gap analyzer

HR/Payroll Agent

- Payroll queries, labour law, NSSF, WCF
- Access to: Employment Act, Labour regulations, NSSF/WCF rules
- Tools: PAYE calculator, NSSF calculator, WCF calculator

Financial Analysis Agent

- Analyzes financial statements, generates insights
- Access to: Financial reporting standards
- Tools: Ratio analyzer, Trend analyzer, Risk assessor

7.3 Multi-Agent Coordination

For complex queries requiring multiple agents:

```

type AgentTask = {
  id: string;
  agentId: string;
  intent: string;
  input: string;
  context: ConversationContext;
  priority: number;
  dependencies: string[]; // Other task IDs this depends on
};

type AgentResult = {
  taskId: string;
  agentId: string;
  output: string;
  confidence: number;
  citations: Citation[];
  requiresHumanReview: boolean;
};

async function coordinateAgents(
  tasks: AgentTask[]
): Promise {
  // Build dependency graph
  // Execute independent tasks in parallel
  // Execute dependent tasks sequentially
  // Merge results
  // Return coordinated response
}

```

7.4 Conversation Memory

Session Memory (Redis)

- Current conversation context (last 20 messages)
- Active entities (company, client, document)
- User preferences for this session

Long-term Memory (PostgreSQL)

- Conversation history (all messages)
- User interaction patterns
- Frequently asked questions
- Company-specific context

Memory Injection

```

type MemoryContext = {
  recentMessages: Message[]; // Last N messages
  companyContext: CompanyProfile; // Current company
  userProfile: UserProfile; // User preferences
  activeEntities: Entity[]; // Current context entities
  conversationSummary?: string; // For long conversations
};

```

8. ERPNext Integration

8.1 Integration Architecture



8.2 ERPNext DocTypes

AIMS Chat

Fields:

- user: Link (User)
- company: Link (Company)
- message_type: Select (user/assistant)
- message: Long Text
- intent: Data
- confidence: Float
- citations: JSON
- agent_used: Data
- created_at: Datetime
- session_id: Data

AIMS Audit Log

Fields:

- user: Link (User)
- company: Link (Company)
- action: Select (query/calculation/document/access)
- input_summary: Text
- output_summary: Text
- sources_used: JSON
- rule_versions: JSON
- risk_level: Select (low/medium/high)
- requires_review: Check
- reviewed_by: Link (User)
- reviewed_at: Datetime

AIMS Knowledge Source

Fields:

- title: Data
- source_type: Select (act/ruling/guidance/policy/upload)
- source_org: Data
- source_url: Data
- document_type: Select
- effective_date: Date
- review_status: Select (pending/approved/rejected)
- reviewed_by: Link (User)
- chunk_count: Int
- last_reviewed: Date

8.3 Context-Aware Responses

When accessed from within ERPNext, the AI has additional context:

```
type ERPNextContext = {  
  currentDocType?: string;      // e.g., "Sales Invoice"  
  currentDocName?: string;      // e.g., "SI-2024-001"  
  currentCompany?: string;      // e.g., "ABC Ltd"  
  userRoles: string[];          // e.g., ["Accounts Manager"]  
  openModules: string[];        // e.g., ["Accounts", "Stock"]  
  recentTransactions?: Transaction[];  
};
```

This allows the AI to provide contextual help:

- "I see you're viewing Sales Invoice SI-2024-001. Would you like me to explain the VAT treatment?"
- "Based on your current Account ledger, I notice..."

8.4 ERPNext Hooks

```
# hooks.py

app_name = "aims_ai"
app_title = "AIMS AI Agent"
app_publisher = "AIMS"
app_description = "AI-powered accounting and tax assistant"
app_version = "0.1.0"

doc_events = {
    "Sales Invoice": {
        "on_submit": "aims_ai.events.on_sales_invoice_submit",
        "on_cancel": "aims_ai.events.on_sales_invoice_cancel",
    },
    "Purchase Invoice": {
        "on_submit": "aims_ai.events.on_purchase_invoice_submit",
    },
    "Journal Entry": {
        "on_submit": "aims_ai.events.on_journal_entry_submit",
    },
}

whitelisted_methods = [
    "aims_ai.api.chat.send_message",
    "aims_ai.api.chat.get_history",
    "aims_ai.api.knowledge.search",
    "aims_ai.api.calculations.calculate",
]

scheduler_events = {
    "daily": [
        "aims_ai.tasks.check_compliance_deadlines",
        "aims_ai.tasks.refresh_knowledge_sources",
    ],
    "weekly": [
        "aims_ai.tasks.generate_compliance_report",
    ],
}
```

9. Database Schema

9.1 Core Tables

```

-- Tenants (Companies)
CREATE TABLE tenants (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name VARCHAR(255) NOT NULL,
  tin VARCHAR(50),
  vrn VARCHAR(50),
  erpnext_site_url VARCHAR(500),
  erpnext_api_key_encrypted TEXT,
  settings JSONB DEFAULT '{}',
  created_at TIMESTAMPTZ DEFAULT NOW(),
  updated_at TIMESTAMPTZ DEFAULT NOW()
);

-- Users
CREATE TABLE users (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id UUID REFERENCES tenants(id),
  erpnext_user_id VARCHAR(100),
  email VARCHAR(255) NOT NULL,
  name VARCHAR(255) NOT NULL,
  role VARCHAR(50) NOT NULL DEFAULT 'viewer',
  language_preference VARCHAR(10) DEFAULT 'en',
  preferences JSONB DEFAULT '{}',
  created_at TIMESTAMPTZ DEFAULT NOW(),
  last_active_at TIMESTAMPTZ
);

-- Conversations
CREATE TABLE conversations (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id UUID REFERENCES tenants(id),
  user_id UUID REFERENCES users(id),
  title VARCHAR(500),
  context JSONB DEFAULT '{}',
  created_at TIMESTAMPTZ DEFAULT NOW(),
  updated_at TIMESTAMPTZ DEFAULT NOW()
);

-- Messages
CREATE TABLE messages (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  conversation_id UUID REFERENCES conversations(id) ON DELETE CASCADE,
  role VARCHAR(20) NOT NULL, -- 'user', 'assistant', 'system'
  content TEXT NOT NULL,
  language VARCHAR(10),
  intent VARCHAR(100),
  agent_used VARCHAR(100),
  confidence FLOAT,
  citations JSONB DEFAULT '[]',
  retrieved_chunks JSONB DEFAULT '[]',
  metadata JSONB DEFAULT '{}',
  created_at TIMESTAMPTZ DEFAULT NOW()
);

-- Knowledge Sources
CREATE TABLE knowledge_sources (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  title VARCHAR(500) NOT NULL,
  source_type VARCHAR(50) NOT NULL,
  source_org VARCHAR(255),
  source_url VARCHAR(1000),
  document_type VARCHAR(100),
  effective_date DATE,
  expiry_date DATE,
  version VARCHAR(50),
  review_status VARCHAR(20) DEFAULT 'pending',
  reviewed_by UUID REFERENCES users(id),
  reviewed_at TIMESTAMPTZ,
  metadata JSONB DEFAULT '{}',
  created_at TIMESTAMPTZ DEFAULT NOW(),
  updated_at TIMESTAMPTZ DEFAULT NOW()
);

```

```

-- Knowledge Chunks (with vector embeddings)
CREATE TABLE knowledge_chunks (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    source_id UUID REFERENCES knowledge_sources(id) ON DELETE CASCADE,
    chunk_index INT NOT NULL,
    content TEXT NOT NULL,
    content_tokens INT,
    language VARCHAR(10),
    section VARCHAR(255),
    page_number INT,
    metadata JSONB DEFAULT '{}',
    embedding VECTOR(1536), -- pgvector
    created_at TIMESTAMPTZ DEFAULT NOW()
);

-- Tax Rules
CREATE TABLE tax_rules (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    rule_code VARCHAR(100) NOT NULL UNIQUE,
    tax_type VARCHAR(50) NOT NULL,
    jurisdiction VARCHAR(100) NOT NULL,
    description TEXT,
    parameters JSONB NOT NULL, -- rates, thresholds, bands
    effective_from DATE NOT NULL,
    effective_to DATE,
    source_source_id UUID REFERENCES knowledge_sources(id),
    version VARCHAR(50) NOT NULL,
    status VARCHAR(20) DEFAULT 'active',
    created_at TIMESTAMPTZ DEFAULT NOW()
);

-- Calculations (Audit Trail)
CREATE TABLE calculations (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    tenant_id UUID REFERENCES tenants(id),
    user_id UUID REFERENCES users(id),
    calculation_type VARCHAR(50) NOT NULL,
    input_parameters JSONB NOT NULL,
    result JSONB NOT NULL,
    rule_version VARCHAR(50) NOT NULL,
    created_at TIMESTAMPTZ DEFAULT NOW()
);

-- Documents (Uploaded)
CREATE TABLE documents (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    tenant_id UUID REFERENCES tenants(id),
    uploaded_by UUID REFERENCES users(id),
    title VARCHAR(500) NOT NULL,
    file_name VARCHAR(500) NOT NULL,
    file_path VARCHAR(1000) NOT NULL,
    file_size BIGINT,
    mime_type VARCHAR(100),
    page_count INT,
    char_count INT,
    tags TEXT[],
    processing_status VARCHAR(20) DEFAULT 'pending',
    processing_error TEXT,
    chunk_count INT DEFAULT 0,
    created_at TIMESTAMPTZ DEFAULT NOW()
);

-- Compliance Calendar
CREATE TABLE compliance_obligations (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    tenant_id UUID REFERENCES tenants(id),
    obligation_type VARCHAR(100) NOT NULL,
    description TEXT NOT NULL,
    due_date DATE NOT NULL,
    frequency VARCHAR(20), -- monthly, quarterly, annual
    tax_type VARCHAR(50),
    status VARCHAR(20) DEFAULT 'pending',
    completed_at TIMESTAMPTZ,
    completed_by UUID REFERENCES users(id),
    evidence_file_id UUID REFERENCES documents(id),

```

```

        created_at TIMESTAMPTZ DEFAULT NOW()
    );

-- Audit Log
CREATE TABLE audit_log (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    tenant_id UUID REFERENCES tenants(id),
    user_id UUID REFERENCES users(id),
    action VARCHAR(100) NOT NULL,
    resource_type VARCHAR(100),
    resource_id VARCHAR(255),
    input_summary TEXT,
    output_summary TEXT,
    sources_used JSONB DEFAULT '[]',
    rule_versions JSONB DEFAULT '[]',
    risk_level VARCHAR(20) DEFAULT 'low',
    ip_address INET,
    user_agent TEXT,
    created_at TIMESTAMPTZ DEFAULT NOW()
);

-- Human Review Queue
CREATE TABLE review_queue (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    tenant_id UUID REFERENCES tenants(id),
    message_id UUID REFERENCES messages(id),
    assigned_to UUID REFERENCES users(id),
    status VARCHAR(20) DEFAULT 'pending',
    priority VARCHAR(20) DEFAULT 'medium',
    risk_level VARCHAR(20),
    reviewer_notes TEXT,
    reviewed_at TIMESTAMPTZ,
    created_at TIMESTAMPTZ DEFAULT NOW()
);

```

9.2 Indexes

```

-- Performance indexes
CREATE INDEX idx_messages_conversation ON messages(conversation_id, created_at);
CREATE INDEX idx_chunks_source ON knowledge_chunks(source_id);
CREATE INDEX idx_chunks_embedding ON knowledge_chunks USING ivfflat (embedding vector_cosine_ops) WITH (lists = 100);
CREATE INDEX idx_chunks_language ON knowledge_chunks(language);
CREATE INDEX idx_chunks_metadata ON knowledge_chunks USING gin(metadata);
CREATE INDEX idx_tax_rules_type ON tax_rules(tax_type, jurisdiction, status);
CREATE INDEX idx_audit_log_tenant ON audit_log(tenant_id, created_at);
CREATE INDEX idx_compliance_due ON compliance_obligations(tenant_id, due_date, status);
CREATE INDEX idx_documents_tenant ON documents(tenant_id, created_at);

```

10. API Contracts

10.1 Chat API

```
// POST /api/v1/chat
type ChatRequest = {
  message: string;
  conversationId?: string;
  context?: {
    erpNext?: ERPNextContext;
    clientProfile?: string;
  };
};

type ChatResponse = {
  conversationId: string;
  messageId: string;
  answer: string;
  confidence: "high" "medium" "low";
  citations: Citation[];
  followUps: string[];
  needsHumanReview: boolean;
  agentUsed: string;
  intent: string;
  language: "en" "sw" "mixed";
};

// Streaming variant
// POST /api/v1/chat/stream
// Returns: text/event-stream with progressive tokens
```

10.2 Knowledge API

```
// GET /api/v1/knowledge/sources
// Query: ?status=approved&type=tax_act&page=1&limit=20

// POST /api/v1/knowledge/sources
// Upload new source document

// POST /api/v1/knowledge/sources/:id/approve
// Admin approval endpoint
```

10.3 Calculations API

```
// POST /api/v1/calculations/payee
// POST /api/v1/calculations/vat
// POST /api/v1/calculations/sdl
// POST /api/v1/calculations/wht
// POST /api/v1/calculations/cit
// POST /api/v1/calculations/provisional-tax

// Each returns:
type CalculationResponse = {
  result: Record;
  ruleVersion: string;
  effectiveDate: string;
  notes: string;
  citations: Citation[];
};
```

10.4 Documents API

```
// GET /api/v1/documents
// POST /api/v1/documents (multipart/form-data)
// DELETE /api/v1/documents/:id
// POST /api/v1/documents/:id/reprocess
```

10.5 ERPNext Integration API

```
// POST /api/v1/integration/erpnext/sync
// POST /api/v1/integration/erpnext/webhook
// GET /api/v1/integration/erpnext/context
```

10.6 Admin API

```
// GET /api/v1/admin/audit-log
// GET /api/v1/admin/review-queue
// POST /api/v1/admin/review-queue/:id/approve
// GET /api/v1/admin/analytics
// GET /api/v1/admin/health
```

11. Security Model

11.1 Authentication

ERPNext Session Authentication

- When accessed from ERPNext, use Frappe session token
- Validate token against ERPNext site
- Map Frappe user to AIMS user

Standalone Authentication

- JWT-based authentication
- Refresh token rotation
- Session timeout (configurable)

11.2 Authorization (RBAC)

RolePermissions

```
viewer  Chat, view calculations, view own history
accountant + Upload documents, run calculations, view client data
tax_consultant + Access tax rules, generate reports, draft documents
admin + Manage sources, approve/reject, manage users
super_admin + System settings, tenant management, audit logs
```

11.3 Data Protection

Encryption

- At rest: AES-256 for sensitive fields (TIN, NIDA, bank details)
- In transit: TLS 1.3
- API keys: Encrypted at rest, decrypted only at use time

Sensitive Data Handling

```
type SensitivityLevel = "public" "internal" "confidential" "restricted";

const sensitivityMap: Record = {
  "tax_type": "internal",
  "rates": "public",
  "tin": "restricted",
  "nida": "restricted",
  "salary": "restricted",
  "bank_account": "restricted",
  "company_name": "confidential",
  "financial_statements": "confidential",
};
```

Chat Log Protection

- Sensitive entities auto-detected and masked in logs
- TIN, NIDA, bank details replaced with [REDACTED]
- Configurable retention policies

11.4 Audit Requirements

Every sensitive action is logged:

- User authentication events
- Data access (who viewed what)
- Calculations performed
- Documents uploaded/downloaded
- Knowledge source changes
- Configuration changes
- Failed authorization attempts

12. Multi-Language Design

12.1 Language Detection

```
type LanguageDetectionResult = {
  primary: "en" | "sw";
  confidence: number;
  isMixed: boolean;
  swahiliRatio: number;
};

function detectLanguage(text: string): LanguageDetectionResult {
  const swahiliIndicators = [
    "nini", "je", "kwa", "la", "ni", "ya", "wa", "na",
    "lini", "imalipwa", "kodi", "biashara", "kampuni",
    "mshahara", "mwajiri", "tarehe", "malipo", "usajili"
  ];

  // Calculate Swahili word ratio
  // Apply confidence threshold
  // Return detection result
}
```

12.2 Bilingual Knowledge Base

All knowledge entries store content in both languages:

```
type BilingualKnowledgeEntry = {
  id: string;
  title: { en: string; sw: string };
  summary: { en: string; sw: string };
  bullets: { en: string[]; sw: string[] };
  content: { en: string; sw: string };
  sourceUrl: string;
  sourceOrg: string;
  effectiveDate?: string;
};
```

12.3 Response Generation

The response composer uses the detected language to:

1. Retrieve language-appropriate chunks
2. Generate response in the detected language
3. Include bilingual citations (source names stay in original)
4. Match follow-up question language

12.4 Swahili Tax Terminology

Maintain a comprehensive terminology mapping:

```
const TAX_TERMS: Record = {
  "value_added_tax": { en: "VAT", sw: "Ongezeko la Thamani" },
  "pay_as_you_earn": { en: "PAYE", sw: "Lipa Kama Unapata" },
  "skills_development_levy": { en: "SDL", sw: "Kodi ya Maendeleo ya Ujuzi" },
  "withholding_tax": { en: "Withholding Tax", sw: "Kodi ya Kukatwa Kokote" },
  "taxpayer_identification": { en: "TIN", sw: "Namba ya Mlipakodi" },
  "tax_return": { en: "Tax Return", sw: "Ripoti ya Kodi" },
  "tax_audit": { en: "Tax Audit", sw: "Ukaguzi wa Kodi" },
  "compliance": { en: "Compliance", sw: "Kufuata Sheria" },
  // ... extensive mapping
};
```

13. Frontend Architecture

13.1 Component Structure

```
app/
├─ layout.tsx           # Root layout
├─ page.tsx             # Landing/redirect
├─ (auth)/
│  ├─ login/page.tsx
│  └─ register/page.tsx
├─ (dashboard)/
│  ├─ layout.tsx       # Dashboard layout with sidebar
│  ├─ page.tsx         # Main dashboard
│  ├─ chat/
│  │  └─ page.tsx      # Chat workspace
│  ├─ calculators/
│  │  └─ page.tsx      # Tax calculators
│  ├─ documents/
│  │  └─ page.tsx      # Document management
│  ├─ compliance/
│  │  └─ page.tsx      # Compliance calendar
│  ├─ knowledge/
│  │  └─ page.tsx      # Knowledge source management
│  ├─ audit/
│  │  └─ page.tsx      # Audit log viewer
│  └─ settings/
│     └─ page.tsx      # User/tenant settings
├─ api/                 # API routes (existing + new)
└─ components/
   ├─ chat/
   │  ├─ ChatWorkspace.tsx
   │  ├─ MessageList.tsx
   │  ├─ MessageBubble.tsx
   │  ├─ Composer.tsx
   │  ├─ CitationCard.tsx
   │  └─ ThinkingIndicator.tsx
   ├─ calculators/
   │  ├─ CalculatorGrid.tsx
   │  ├─ PayeCalculator.tsx
   │  ├─ VatCalculator.tsx
   │  ├─ SdlCalculator.tsx
   │  ├─ WhtCalculator.tsx
   │  └─ ResultCard.tsx
   ├─ documents/
   │  ├─ DocumentList.tsx
   │  ├─ UploadZone.tsx
   │  └─ DocumentCard.tsx
   ├─ knowledge/
   │  ├─ SourceList.tsx
   │  ├─ SourceCard.tsx
   │  └─ ReviewQueue.tsx
   ├─ layout/
   │  ├─ Sidebar.tsx
   │  ├─ Header.tsx
   │  └─ Breadcrumb.tsx
   └─ shared/
      ├─ Button.tsx
      ├─ Card.tsx
      ├─ Modal.tsx
      └─ Toast.tsx
```

13.2 State Management

- **Server State:** React Query / SWR for API data
- **Client State:** Zustand for UI state
- **Form State:** React Hook Form
- **WebSocket:** For streaming LLM responses

13.3 Streaming Responses

```
// Client-side streaming
async function streamChat(message: string, onToken: (token: string) => void) {
  const response = await fetch("/api/v1/chat/stream", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ message }),
  });

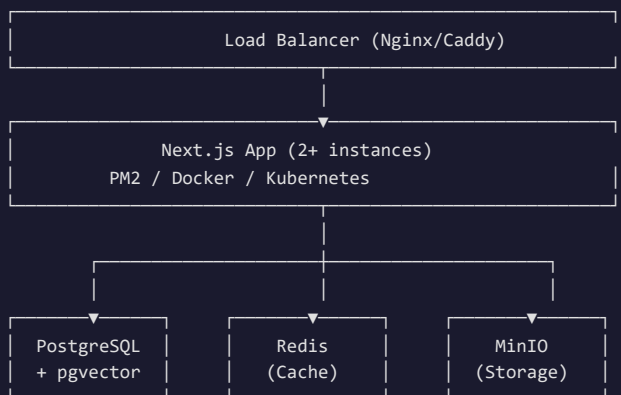
  const reader = response.body?.getReader();
  const decoder = new TextDecoder();

  while (true) {
    const { done, value } = await reader!.read();
    if (done) break;

    const chunk = decoder.decode(value);
    // Parse SSE events
    // Extract tokens
    // Call onToken for each
  }
}
```

14. Infrastructure

14.1 Deployment Architecture



14.2 Environment Configuration

```
# Database
DATABASE_URL=postgresql://user:pass@localhost:5432/aims
REDIS_URL=redis://localhost:6379
```

```
OPENAI_API_KEY=sk-...
OPENAI_MODEL=gpt-4o
OPENAI_EMBEDDING_MODEL=text-embedding-3-small
```

```
ERPNEXT_SITE_URL=https://erp.yourcompany.com
ERPNEXT_API_KEY=...
ERPNEXT_API_SECRET=...
```

```
S3_ENDPOINT=http://localhost:9000
S3_BUCKET=aims-documents
S3_ACCESS_KEY=...
S3_SECRET_KEY=...
```

```
JWT_SECRET=...
SESSION_TIMEOUT=3600
ENCRYPTION_KEY=...
```

```
ENABLE_OCR=true
ENABLE_STREAMING=true
MAX_UPLOAD_SIZE_MB=50
```

14.3 Monitoring

Health Checks

```
// GET /api/health
type HealthCheck = {
  status: "healthy" | "degraded" | "unhealthy";
  version: string;
  uptime: number;
  checks: {
    database: "ok" | "error";
    redis: "ok" | "error";
    llm: "ok" | "error";
    vectorStore: "ok" | "error";
  };
};
```

Metrics to Monitor

- Response latency (p50, p95, p99)
- RAG retrieval accuracy
- LLM token usage
- Error rates by endpoint
- Active conversations
- Knowledge base size
- Cache hit rates

15. Migration Path

15.1 Phase 1: Foundation (Current → MVP)

What exists now:

- ☒ Next.js 15 app shell
- ☒ Basic chat UI
- ☒ Static knowledge base (9 entries)
- ☒ Keyword retrieval with synonyms
- ☒ PAYE, VAT, SDL, WHT calculators
- ☒ PDF upload and extraction
- ☒ Swahili detection

What to add in Phase 1:

1. PostgreSQL + pgvector setup
2. Move knowledge base to database
3. Add OpenAI embeddings generation
4. Replace keyword search with vector search + BM25 hybrid
5. Add OpenAI GPT-4 integration for response generation
6. Add streaming responses
7. Basic conversation history (database)
8. Environment configuration

15.2 Phase 2: Core Intelligence

1. Intent classification system
2. Agent orchestrator (single agent first)
3. Citation extraction and verification
4. Confidence scoring
5. Human review queue
6. Tax rule versioning system
7. Enhanced calculators (CIT, Provisional Tax)
8. Document understanding pipeline

15.3 Phase 3: ERPNext Integration

1. Frappe custom app development
2. ERPNext DocType definitions
3. Session synchronization
4. Context-aware responses
5. Event-driven triggers
6. Permission alignment

15.4 Phase 4: Multi-Agent System

1. Agent framework development
2. Specialized agents (Tax, Accounting, ERP, Compliance)
3. Multi-agent coordination
4. Specialized tools per agent
5. Agent performance monitoring

15.5 Phase 5: Enterprise Features

1. Multi-tenant architecture
2. Advanced RBAC
3. Compliance calendar
4. Document drafting engine
5. Financial analysis engine
6. Audit trail and reporting
7. WhatsApp/SMS integration
8. Mobile app

15.6 Phase 6: Advanced AI

1. Fine-tuned models for Tanzania tax
2. Advanced document understanding (OCR, table extraction)
3. Predictive analytics

- 4. Anomaly detection
- 5. Automated reporting

Appendix A: File Structure (Target)

```

aims-ai-agent/
├── app/                                # Next.js App Router
│   ├── (auth)/
│   │   ├── login/page.tsx
│   │   └── layout.tsx
│   ├── (dashboard)/
│   │   ├── layout.tsx
│   │   ├── page.tsx
│   │   ├── chat/page.tsx
│   │   ├── calculators/page.tsx
│   │   ├── documents/page.tsx
│   │   ├── compliance/page.tsx
│   │   ├── knowledge/page.tsx
│   │   └── settings/page.tsx
│   ├── api/
│   │   ├── v1/
│   │   │   ├── chat/route.ts
│   │   │   ├── chat/stream/route.ts
│   │   │   ├── calculations/
│   │   │   ├── documents/
│   │   │   ├── knowledge/
│   │   │   ├── integration/
│   │   │   ├── admin/
│   │   └── health/route.ts
│   ├── auth/
│   ├── layout.tsx
│   └── globals.css
├── lib/                                # Core library code
│   ├── agents/                         # Agent definitions
│   │   ├── orchestrator.ts
│   │   ├── intent-router.ts
│   │   ├── context-manager.ts
│   │   ├── tax-agent.ts
│   │   ├── accounting-agent.ts
│   │   ├── erp-agent.ts
│   │   ├── compliance-agent.ts
│   │   ├── document-agent.ts
│   │   └── audit-agent.ts
│   ├── rag/                           # RAG pipeline
│   │   ├── query-processor.ts
│   │   ├── retriever.ts
│   │   ├── reranker.ts
│   │   ├── context-assembler.ts
│   │   ├── embedding-service.ts
│   │   └── citation-extractor.ts
│   ├── llm/                           # LLM integration
│   │   ├── client.ts
│   │   ├── prompts.ts
│   │   ├── streaming.ts
│   │   └── token-counter.ts
│   ├── knowledge/                     # Knowledge management
│   │   ├── ingestion-pipeline.ts
│   │   ├── chunker.ts
│   │   ├── quality-checker.ts
│   │   └── source-manager.ts
│   ├── calculators/                  # Tax calculators
│   │   ├── paye.ts
│   │   ├── vat.ts
│   │   ├── sdl.ts
│   │   ├── wht.ts
│   │   ├── cit.ts
│   │   └── rule-engine.ts
│   ├── integrations/                 # External integrations
│   │   ├── erpNext/
│   │   │   ├── client.ts
│   │   │   ├── sync.ts
│   │   │   └── context.ts
│   │   ├── storage/
│   │   │   └── s3-client.ts
└── db/                               # Database layer

```

```

├── client.ts
├── migrations/
├── repositories/
├── auth/                # Authentication
├──   ├── jwt.ts
├──   ├── session.ts
├──   └── permissions.ts
├── security/            # Security utilities
├──   ├── encryption.ts
├──   ├── sanitization.ts
├──   └── audit.ts
├── i18n/                # Internationalization
├──   ├── language-detector.ts
├──   ├── terminology.ts
├──   └── translations.ts
├── types/               # TypeScript types
├──   ├── index.ts
├──   ├── chat.ts
├──   ├── knowledge.ts
├──   ├── erpNext.ts
├──   └── agents.ts
├── components/          # React components
├──   ├── chat/
├──   ├── calculators/
├──   ├── documents/
├──   ├── knowledge/
├──   ├── layout/
├──   └── shared/
├── hooks/               # React hooks
├──   ├── useChat.ts
├──   ├── useCalculations.ts
├──   ├── useDocuments.ts
├──   └── useStreaming.ts
├── prisma/              # Database schema
├──   └── schema.prisma
├── scripts/             # Build/deploy scripts
├──   ├── seed-knowledge.ts
├──   ├── generate-embeddings.ts
├──   └── migrate.ts
├── docs/                # Documentation
├──   ├── ARCHITECTURE.md
├──   ├── API.md
├──   ├── DEPLOYMENT.md
├──   └── USER-GUIDE.md
├── tests/               # Test files
├──   ├── unit/
├──   ├── integration/
├──   └── e2e/
├── erpNext-app/         # Frappe custom app
├──   ├── aims_ai/
├──   │   ├── doctype/
├──   │   ├── api/
├──   │   ├── hooks.py
├──   │   └── tasks.py
├──   └── setup.py
├── package.json
├── tsconfig.json
├── next.config.ts
├── tailwind.config.ts
├── docker-compose.yml
├── Dockerfile
└── .env.example

```

Appendix B: Key Decisions Log

DecisionChoiceRationale

Primary DB PostgreSQL + pgvector Single DB for relational + vector, reduces complexity
LLM Provider OpenAI (primary), configurable Best balance of cost/performance for English + Swahili
Embedding Model text-embedding-3-small Cost-effective, good multilingual support
Framework Next.js 15 (App Router) Already in use, good DX, SSR support
ERPNext Integration Custom Frappe app Native integration, proper permission model
Caching Redis Fast, supports sessions + pub/sub
Object Storage MinIO (S3-compatible) Self-hosted option, S3-compatible
Message Queue BullMQ (Redis-backed) Simple, Redis-based, good for background jobs
Auth JWT + ERPNext session Dual auth for standalone + ERPNext embedded
Streaming Server-Sent Events Simpler than WebSockets for LLM streaming

This architecture document is a living document. It will be updated as implementation progresses and requirements evolve.